



DevContext AI

IQOO CITY BATTLES 2026

Your project's brain, built before you code.

A phone-first architecture blueprinting canvas. Speak an idea or photograph a whiteboard, and an on-device model turns it into a vetted, persistent project scaffold — delivered straight into your IDE.

Track 06 Developer Tools

City Battles + National Grand Finale eligible

Chennai City Battle 12-13 Sept 2026

The most expensive hour of any project is the first one.

It is spent in browser tabs, not in an editor — and nothing survives it.



The scaffold loop

Hours lost bouncing between GitHub, docs and an empty editor just to decide on a structure.



Blind dependency picks

Keyword search ranks READMEs and stars. It cannot tell you a library has been dead for a year.



Unread community signal

The Reddit thread that already solved your problem is never opened during planning.



Nothing persists

Every decision and its reasoning dies with the closed tab. The next developer starts blind.

The gap: tooling helps you write code. Almost nothing helps you decide what to write — and nothing records **why** you decided it.

The developer with no architect to ask.

Early-career engineers at Indian IT service providers

TCS, Infosys, Wipro, Cognizant. Handed a new project, expected to stand up modern full-stack scaffolding from scratch, with minimal access to a senior system architect.

Tier-2 and Tier-3 college engineering students

Building their first real projects with no mentor to review architecture choices, and no way to tell a maintained library from an abandoned one.

Hundreds

of hours lost to clashing boilerplate and misconfigured dependencies

Zero

architectural review before the first commit lands

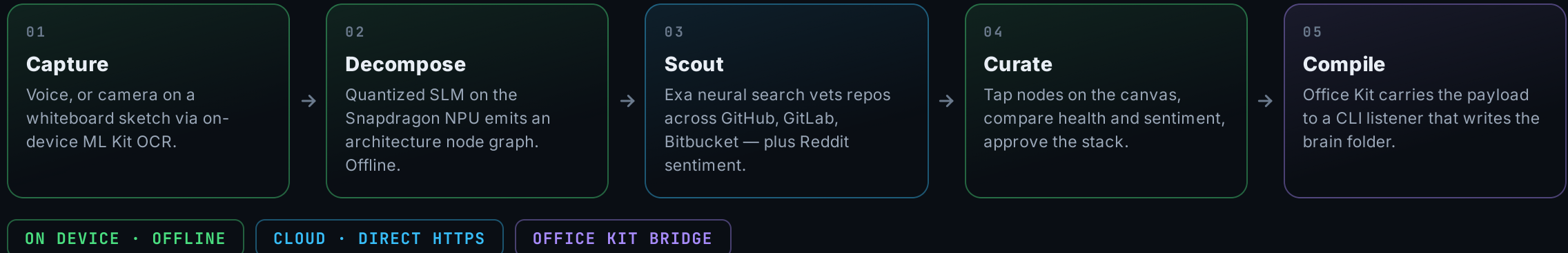
Off-desk

where the decisions actually happen: standups, commutes, whiteboards

Why phone-first: the **blueprint** phase is already mobile, even when the build is not. We meet the decision where it happens instead of forcing it back to a desk.

Five components. One pass. No backend.

Scope was deliberately cut from fifteen modules to five so it ships inside 19 on-site hours.



Deliberate omission: there is no orchestration backend. The client calls Exa directly, because a localhost dependency cannot function during Red Light — **55% of the on-site build window.**

The brain never leaves the phone.

Already proven on a laptop. Verbatim model output is committed in the repository.

5/5

requests produced schema-valid
node graphs

1st try

every one. The JSON repair loop
never had to fire

Offline

confirmed against a fully blocked
network

70

automated checks green across two
suites

```
# quantized Phi-3-mini int4 · onnxruntime-genai · no network
$ python -m devcontext.decompose --suite
```

[1/5] audio streaming, offline sync	VALID	4 nodes
[2/5] telemedicine booking, payments	VALID	4 nodes
[3/5] cli photo dedupe, perceptual hash	VALID	2 nodes
[4/5] offline attendance, tier-2 college	VALID	3 nodes
[5/5] realtime quiz game, anti-cheat	VALID	4 nodes

schema-valid: 5/5 – suite green



492 seconds on laptop CPU

That average is the entire case for the on-site work, not a footnote. A 3.8B int4 model on CPU is nowhere near interactive.

NPU acceleration is what makes this loop usable on the phone — and it is the first Green Light task in Chennai.

It leaves something behind.

Every other tool closes a tab. This one writes a folder that outlives the session.

```
my-new-project/
├── .devcontext/
│   ├── context.json    the project brain
│   ├── semantic_cache.db  what was chosen, and why
│   ├── open_source_maps/ portable vector store
│   └── trends_report.md  vetted tooling per node
└── init.sh             community risk log
                        one command bootstrap
```

```
$ bash init.sh
[clone] rxdb           → reference/
[clone] fastify        → reference/
[deps]  Node detected → npm install
```

context.json is the point

A structural ledger recording not just the stack but the reasoning. It stays in the repo as a permanent brief for the next teammate — or the next AI coding agent.

Rationale is composed from evidence

Never generated by the model. A hallucinated justification in a file that outlives the project is worse than none at all.

Search results are untrusted input

init.sh shallow-clones vetted repos read-only into reference/ and resolves dependencies only from your own manifests. No blind installs.

Office Kit is the product, not a checkbox.

The handoff generates bridge activity because it **is** the core loop — nothing was bolted on for telemetry.



Shared clipboard

The compiled node-graph payload crosses as structured JSON, phone to laptop.



File transfer

Larger payloads and build bundles move by drag-and-drop across the bridge.



Remote control

Laptop keyboard drives phone fields through screen mirroring during Red Light.

```
PHONE compiles payload → OFFICE KIT clipboard + file → CLI LISTENER writes .devcontext/
```

Precise about the mechanism: the bridge transports the payload; the CLI listener performs the filesystem write. A clipboard cannot create a directory, and we do not claim it does.

What already works, and what does not.

All of the below is public in the submitted repository.

COMPONENT	STATUS	EVIDENCE
Decompose	BUILT	Quantized Phi-3-mini int4 via onnxruntime-genai. 5/5 schema-valid, all first-attempt. Offline verified against a blocked network. Verbatim output committed.
Compile	BUILT	Full .devcontext/ tree plus a syntax-checked init.sh. 34 automated checks.
Scout	WIRED	Exa integration complete against the published contract, unit-tested on synthetic responses. Live API run pending a key.
Capture	ON-SITE	Camera OCR via ML Kit and microphone STT. Scheduled Saturday Red Light.
Curate	ON-SITE	Flutter node-graph canvas. Scheduled first Green Light window.
NPU runtime	ON-SITE	onnxruntime-genai Android bindings. Highest-risk task, with a documented fallback and a decision deadline.

Why show the gaps: a jury can verify every claim above in the repo. We would rather demo five components that fully work than describe fifteen that do not.

19 hours, planned around the format.

Red Light is 55% of the build. Everything below is sequenced so no Red hour needs an unrestricted laptop.

GREEN • 8.5 H dual device

Flutter scaffold and Android toolchain

adb reverse hot-reload set up before the first Red window

ONNX Runtime Android bridge and first on-device inference

NPU delegate configuration · heavy compiles · repo pushes

RED • 10.5 H phone only, via Office Kit

Node canvas built through live reload on the mirrored screen

Camera OCR and microphone capture on the loaner device

On-device prompt tuning at real context limits

Office Kit handoff testing · demo rehearsal

Sat 19:00

Checkpoint: canvas, camera OCR and voice live on the phone

Sun 01:00

Hard decision point on the NPU bridge, with a documented fallback

Sun 09:00

Full loop: speak an idea, brain folder lands on the laptop

Pre-loaded: the 2.73 GB quantized model is already on disk. It took 3 h 21 min to fetch and will not be pulled over event wifi.

Production frontend, plus low-level automation.

MohanaMantra 2K26 portal

Production web infrastructure for Mohan Babu University's national cultural festival. Asset preloading pipelines, Zustand state architecture, GSAP and Framer Motion UI. Passed RapidScan audits across 80+ vulnerability vectors with zero flaws.

AIStream research suite

Localized Streamlit application wrapping multi-modal research tools, structuring external documentation through automated semantic index routing.

ArduPilot navigation modules

Autonomous drone workspace modules in WSL/Ubuntu. Bash compilation workflows and vector trajectory alignment in simulation.

We showed up with the hard part already solved.

The local inference pipeline, the JSON schema contract, the repair loop and the artifact generator were built and verified **before** this submission. Chennai is for the phone layer, not for discovery. That is deliberate: it moves the highest-uncertainty work off the clock, and it is why we cut scope from fifteen modules to five before writing a line of code.

Repo github.com/sameerreddy789/IQOO_Hackathon-

Track 06 · Developer Tools · Finale eligible